NETWORK TRAFFIC ANOMALY DETECTION AI

PERPARED BY:

EL MOKHTAR FERTIT

SAID LOUHAM

SUPERVISED BY:

PROFESSEUR: SOUFIANE HAMIDA

ACADEMIC YEAR:

2025/2026

I. Chapter 1: Introduction

1. Background:

With the rapid growth of computer networks and internet-based services, network infrastructures have become essential in daily activities, business operations, education, and communication. However, this expansion has also increased cybersecurity risks and exposed systems to various types of network attacks and abnormal behaviors.

Traditional security solutions such as firewalls and signature-based intrusion detection systems are effective for detecting previously known threats, but they often face difficulties when identifying unknown or evolving attack patterns. As modern cyberattacks become more complex and dynamic, intelligent approaches are required to improve network monitoring and threat detection.

Artificial Intelligence (AI) and Machine Learning (ML) provide new opportunities for cybersecurity by enabling systems to automatically analyze traffic patterns and detect unusual activities. Among these techniques, anomaly detection plays an important role because it focuses on identifying behaviors that differ from normal network activity.

This project proposes a network traffic anomaly detection system using clustering techniques to automatically identify suspicious traffic patterns from network data and support proactive security analysis.

2. Problem Statement:

Network environments generate a large amount of traffic data continuously, making manual monitoring difficult and inefficient. Traditional detection approaches often depend on predefined signatures and known attack behaviors, which limits their ability to detect new or unknown threats.

In addition, real-world network traffic contains complex patterns and high-dimensional data, making anomaly identification challenging.

The problem addressed in this project is how to design and implement an intelligent system capable of analyzing network traffic and detecting anomalous behaviors automatically using machine learning techniques without relying entirely on predefined attack signatures.

3. Project Objectives

The main objective of this project is to develop an AI-based system capable of detecting abnormal network traffic using clustering methods.

The specific objectives are:

- Collect and prepare network traffic data.

- Perform data preprocessing and cleaning.
- Explore the dataset using Exploratory Data Analysis (EDA).
- Select and transform relevant features.
- Train a clustering model for anomaly detection.
- Evaluate model performance using clustering metrics.
- Develop a web application to allow users to analyze network traffic and visualize results.
- Improve understanding of network behavior through graphical analysis.

4. Project Scope

This project focuses on anomaly detection in network traffic using unsupervised machine learning techniques.

The implemented system includes:

- Network dataset processing.
- Data cleaning and preparation.
- Feature engineering.
- Clustering-based anomaly detection.
- Model evaluation.
- Interactive web interface for prediction and visualization.

The project does not include:

- Real-time packet capture.
- Deep packet inspection.
- Integration with production security infrastructure.
- Automatic incident response.

5. Methodology Overview

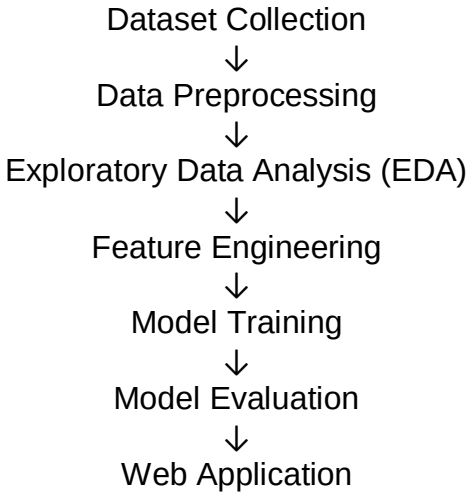
The project follows a machine learning workflow composed of several stages.

First, network traffic data is collected and loaded into the analysis environment. Then preprocessing operations are applied to clean and prepare the data. Exploratory Data Analysis is performed to understand feature distributions and identify patterns.

After preparation, feature engineering techniques are applied before training the clustering model. The trained model is evaluated using appropriate clustering metrics to measure performance.

Finally, a web application is developed to provide an interface for interaction with the anomaly detection system and visualization of the generated results.

Workflow:



6. Report Organization

This report is organized into five chapters:

- **Chapter 1:** Introduction and project overview.
- **Chapter 2:** Literature review and theoretical background.
- **Chapter 3:** Methodology and implementation process.
- **Chapter 4:** Experimental results, evaluation, and discussion.
- **Chapter 5:** Conclusion and future improvements.

7. Chapter Conclusion

This chapter introduced the context and motivation behind network traffic anomaly detection and presented the objectives, scope, and methodology of the proposed system. The next chapter presents the theoretical foundations and related work necessary to understand anomaly detection and machine learning techniques used in cybersecurity.

II. Chapter 2: Literature Review

1. Introduction

Network traffic analysis has become an important area in cybersecurity due to the increasing number of cyberattacks. Traditional detection methods often struggle to detect new and unknown threats. Machine Learning offers intelligent approaches that improve the ability to identify abnormal network behavior automatically.

This chapter presents the main concepts related to network traffic analysis, anomaly detection, machine learning, and clustering techniques used in this project.

2. Network Traffic Analysis

Network traffic analysis is the process of monitoring and examining data exchanged across a network to understand communication behavior and identify suspicious activities.

Its objectives include:

- Monitoring network activity
- Detecting attacks
- Improving security
- Identifying abnormal behavior
- Network traffic is generally classified into:
- Normal traffic → legitimate communication
- Abnormal traffic → suspicious or malicious activity

3. Anomaly Detection

Anomaly detection aims to identify activities that differ from normal network behavior.

Unlike traditional signature-based approaches, anomaly detection can detect unknown threats by analyzing patterns in data.

Advantages:

- Detects unknown attacks
- Adapts to changing environments
- Limitations:
- Can generate false positives
- Requires proper data preparation

4. Machine Learning in Cybersecurity

Machine Learning allows systems to learn patterns from data and support automated security decisions.

Common applications:

- Intrusion detection
- Malware detection

- Behavioral analysis
- Machine Learning methods include:
- Supervised Learning

Unsupervised Learning
Reinforcement Learning
This project uses unsupervised learning.

5. Clustering for Anomaly Detection

Clustering groups similar observations together and separates different behaviors into clusters.
Among clustering methods, K-Means is widely used due to its simplicity and efficiency.
Main steps:

- Select number of clusters
- Assign data points
- Update cluster centers
- Repeat until convergence

6. Evaluation Metrics

To evaluate clustering performance, the following metrics were used:

- Silhouette Score → Measures cluster quality.
- Davies–Bouldin Index → Measures separation between clusters.
- Adjusted Rand Index (ARI) → Measures agreement between predicted clusters and reference labels.

7. Related Work

Previous studies showed that clustering techniques can effectively identify unusual network behaviors and support intrusion detection systems.
This project applies clustering methods to detect anomalies and provides result visualization through a web application.

III. Chapter 3 — Methodology and Implementation

1. Introduction

This chapter presents the methodology used to develop the Network Traffic Anomaly Detection system. The implementation follows an unsupervised machine learning workflow including data loading, exploratory analysis, preprocessing, clustering, evaluation, and deployment through a Streamlit web application.

2. Dataset Description

The project uses the **NSL-KDD** dataset, an improved version of the KDD Cup 1999 dataset for intrusion detection.

Two files were used:

File	Rows	Purpose
KDDTrain+.txt	125,973	Model training
KDDTest+.txt	22,544	Evaluation

Each record contains:

- 41 network traffic features
- 1 attack label
- 1 difficulty value

Important categorical features:

- protocol_type
- service
- flag

Dataset files were stored inside:

- data/raw/

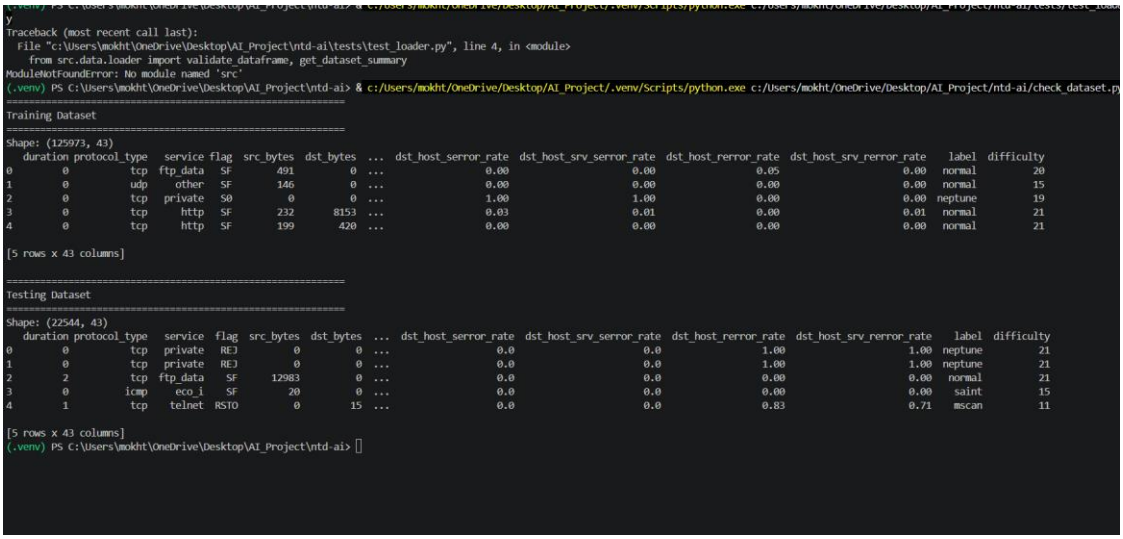


Figure 3.1 Dataset Preview

3. Exploratory Data Analysis (EDA)

EDA was performed to understand dataset characteristics before model training.
The analysis included:

- Dataset dimensions
- Missing values detection
- Duplicate checking
- Attack distribution
- Feature distributions
- Correlation analysis

Observed results:

- Training shape: (125973, 43)
- Test shape: (22544, 43)
- 23 attack classes detected
- No missing values found

Add:

- Distribution plots
- Correlation heatmap

4. Data Preprocessing

Before clustering, preprocessing was applied.
Processing steps:

- Remove label, binary_label, and difficulty.
- Save labels separately for evaluation.
- Encode categorical features using OneHotEncoder.
- Scale numerical features using StandardScaler.
- Fit preprocessing on training data.
- Transform both datasets.

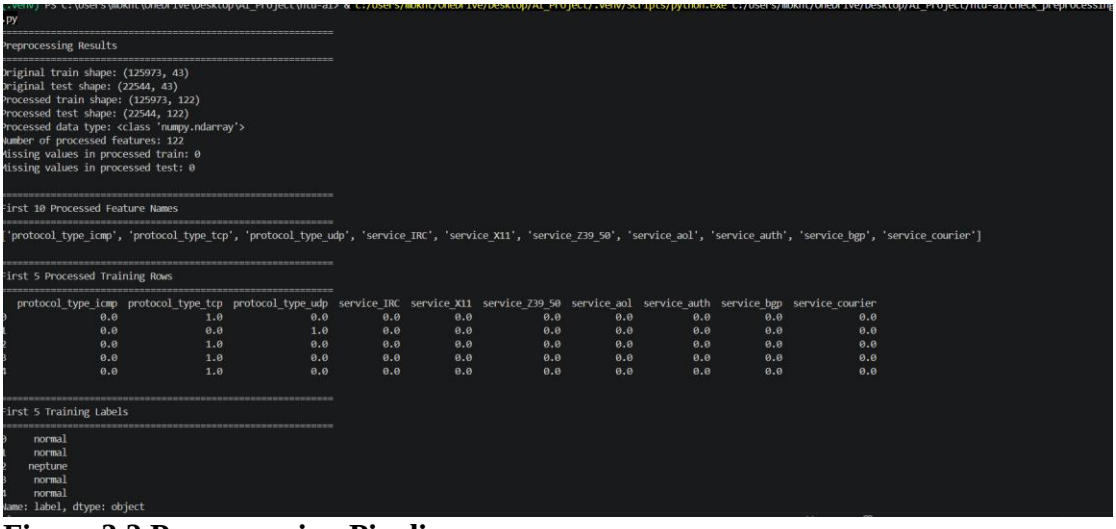


Figure 3.2 Preprocessing Pipeline

5. K-Means Clustering

K-Means was selected to group traffic records according to behavior.
Several values of k were tested.

K	Silhouette	DB Index
3	0.4203	1.0628
4	0.4254	0.9609
5	0.4404	0.8884

The final model used **K = 5** because it produced:

- Highest Silhouette Score
- Lowest Davies–Bouldin Index
- Better cluster separation

The labels were not used during training and were only used afterward for interpretation.

6. Model Evaluation

The clustering model was evaluated using:

Metric	Purpose
Silhouette Score	Cluster quality
Davies–Bouldin Index	Cluster separation
Adjusted Rand Index	Label agreement
Precision	Anomaly accuracy
Recall	Attack detection
F1 Score	Overall balance

```
============================================================
Evaluation Dataset
============================================================
Test rows: 22544
Processed shape: (22544, 122)
Number of clusters: 5

============================================================
Cluster Counts
============================================================
cluster
0    14611
1     2106
2        2
3     5183
4      642
Name: count, dtype: int64

============================================================
Clustering Metrics
============================================================
Silhouette Score: 0.3389
Davies-Bouldin Index: 0.9794
Adjusted Rand Index: 0.1793

============================================================
Cluster vs Binary Label
============================================================
binary_label  attack  normal
cluster
0               5040    9571
1               2101       5
2                  0       2
3               5095      88
4                597      45
```

Figure 3.3 Evaluation Results

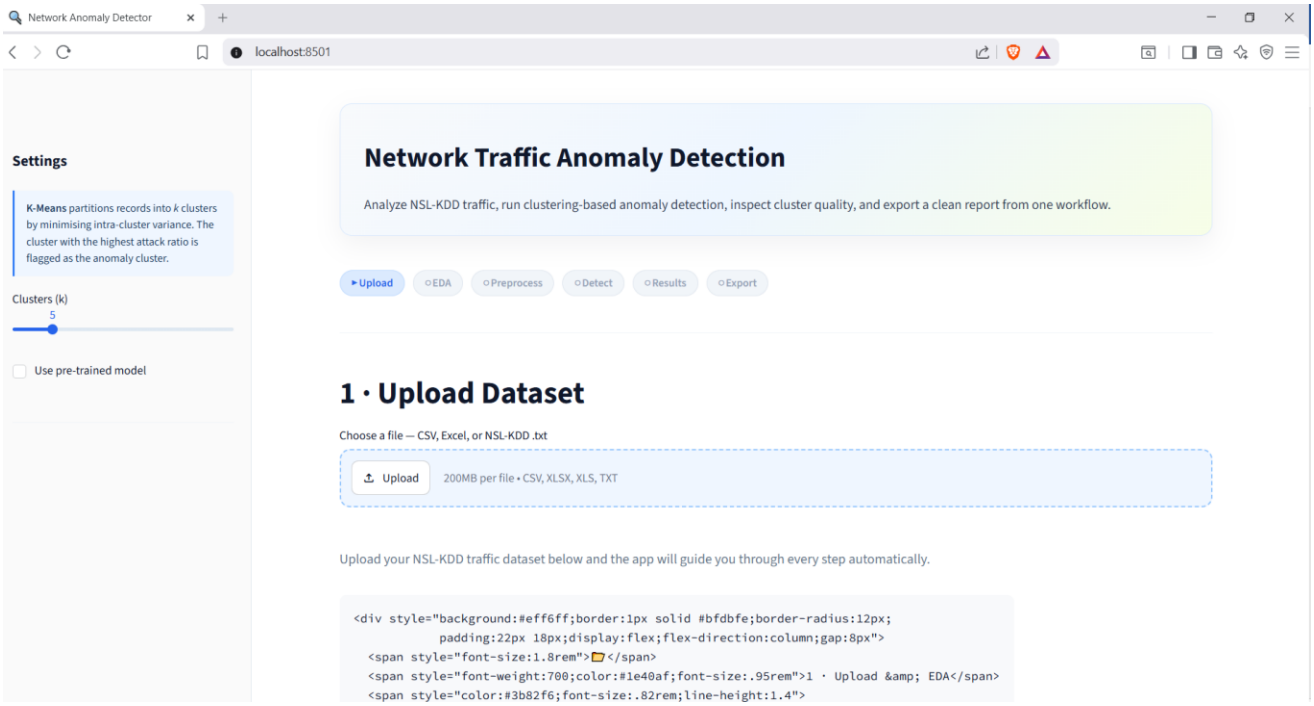
7. Streamlit Application

A Streamlit application was developed to provide an interactive workflow.
Application steps:

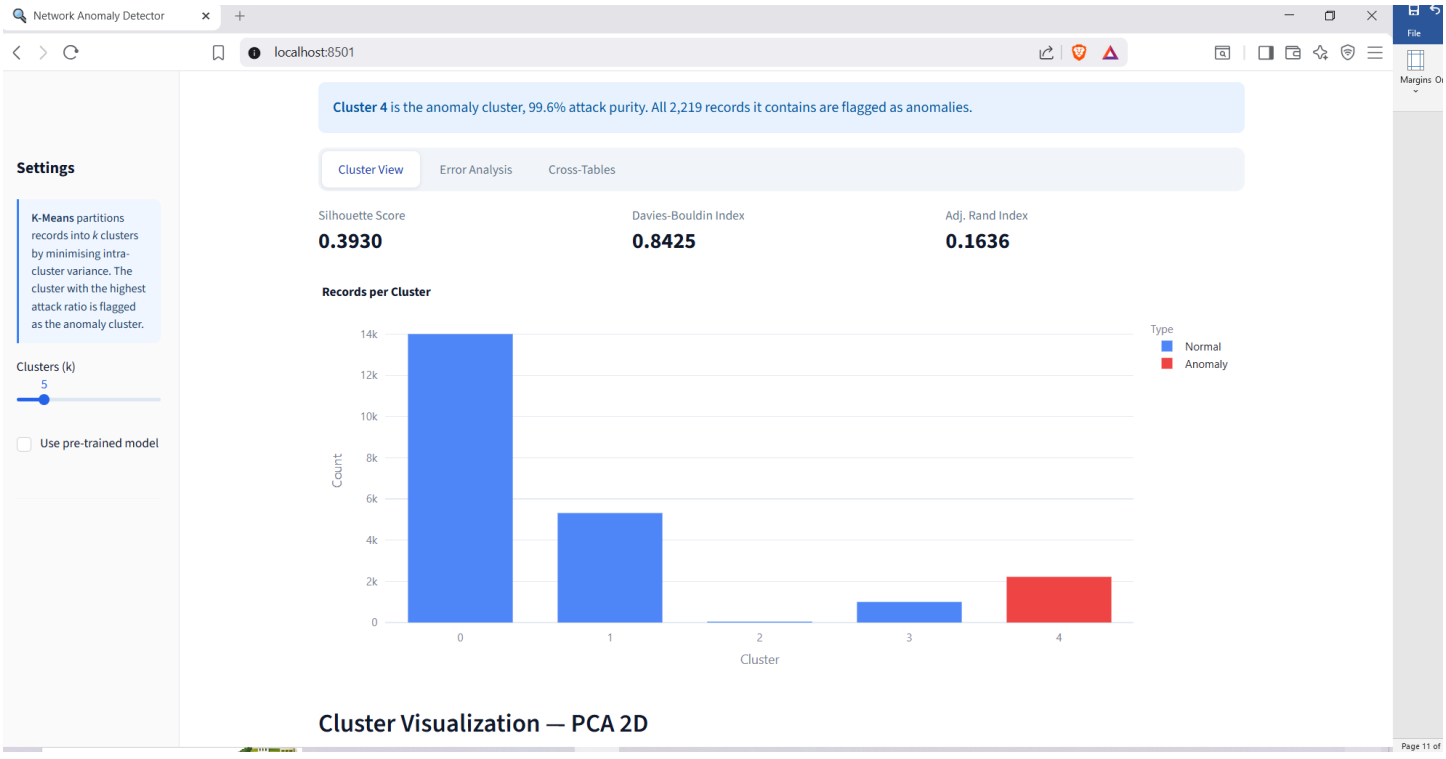
- Upload dataset
- Explore EDA
- Preprocess data
- Run clustering
- Visualize anomalies
- Export results

Additional features:

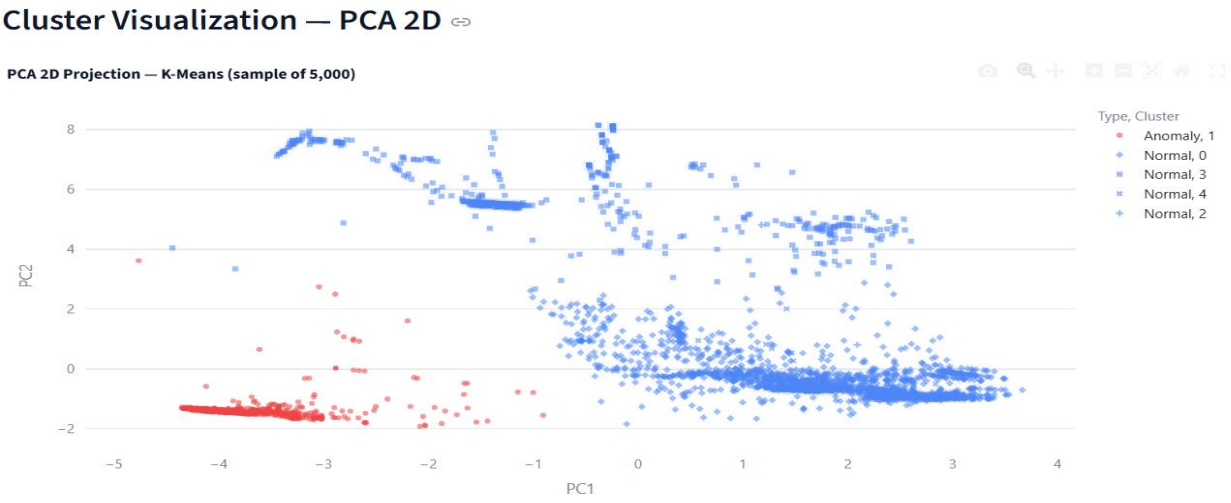
- PCA visualization
- Export CSV/PDF
- Save analysis history
- Use pre-trained model



Results Page



Cluster Visualization



8. Testing

The system was validated using automated tests.
Covered modules:

- Data loader
- Preprocessing
- K-Means model
- Evaluation metrics

Testing command:
python -m pytest

```
testpaths: tests
plugins: anyio-4.13.0
collected 37 items

tests/test_loader.py::test_validate_empty_dataframe PASSED                [  2%]
tests/test_loader.py::test_validate_too_few_columns PASSED                [  5%]
tests/test_loader.py::test_validate_entirely_null_column PASSED           [  8%]
tests/test_loader.py::test_validate_valid_nslkdd PASSED                   [ 10%]
tests/test_loader.py::test_summary_row_count PASSED                       [ 13%]
tests/test_loader.py::test_summary_col_count PASSED                       [ 16%]
tests/test_loader.py::test_summary_null_count_clean PASSED                [ 18%]
tests/test_loader.py::test_summary_detects_label_column PASSED            [ 21%]
tests/test_loader.py::test_summary_no_label_column PASSED                 [ 24%]
tests/test_metrics.py::test_binary_labels_normal PASSED                   [ 27%]
tests/test_metrics.py::test_binary_labels_all_normal PASSED               [ 29%]
tests/test_metrics.py::test_evaluate_has_all_keys PASSED                  [ 32%]
tests/test_metrics.py::test_silhouette_in_valid_range PASSED              [ 35%]
tests/test_metrics.py::test_all_noise_returns_none PASSED                 [ 37%]
tests/test_metrics.py::test_no_labels_gives_none_ari PASSED               [ 40%]
tests/test_metrics.py::test_single_cluster_returns_none PASSED            [ 43%]
tests/test_metrics.py::test_identifies_highest_attack_ratio PASSED        [ 45%]
tests/test_metrics.py::test_no_attack_column_returns_first PASSED         [ 48%]
tests/test_metrics.py::test_perfect_detection PASSED                      [ 51%]
tests/test_metrics.py::test_zero_detection PASSED                         [ 54%]
tests/test_metrics.py::test_missed_attack_types_populated PASSED          [ 56%]
tests/test_metrics.py::test_error_analysis_keys PASSED                    [ 59%]
tests/test_metrics.py::test_cluster_label_table_shape PASSED              [ 62%]
tests/test_models.py::test_kmeans_returns_fitted_model PASSED             [ 64%]
tests/test_models.py::test_kmeans_predict_shape PASSED                    [ 67%]
tests/test_models.py::test_kmeans_predict_valid_labels PASSED             [ 70%]
tests/test_models.py::test_kmeans_separates_two_clusters PASSED           [ 72%]
tests/test_models.py::test_get_anomaly_mask_dtype PASSED                  [ 75%]
tests/test_models.py::test_get_anomaly_mask_count PASSED                  [ 78%]
tests/test_preprocessor.py::test_output_is_ndarray PASSED                 [ 81%]
tests/test_preprocessor.py::test_row_count_preserved PASSED               [ 83%]
tests/test_preprocessor.py::test_onehot_expands_columns PASSED            [ 86%]
tests/test_preprocessor.py::test_no_nan_after_transform PASSED            [ 89%]
tests/test_preprocessor.py::test_labels_preserved PASSED                  [ 91%]
tests/test_preprocessor.py::test_feature_names_match_columns PASSED       [ 94%]
tests/test_preprocessor.py::test_transform_with_existing_shape PASSED     [ 97%]
tests/test_preprocessor.py::test_label_column_dropped PASSED              [100%]

============================================== 37 passed in 4.92s ==============================================

(.venv) C:\Users\mokht\OneDrive\Desktop\AI_Project\ntd-ai>
```

IV. Chapter 4 — Results and Discussion

1. Introduction

This chapter presents the experimental results obtained from the network traffic anomaly detection system. The objective is to evaluate clustering performance, analyze detected patterns, and demonstrate the functionality of the Streamlit application.

2. Clustering Results

Several K-Means configurations were tested to determine the most suitable number of clusters.

Number of Clusters	Silhouette Score	Davies–Bouldin Index
3	0.4203	1.0628
4	0.4254	0.9609
5	**0.4404**	**0.8884**

The best results were obtained with **k = 5**.

This configuration achieved:

- Higher cluster quality
- Better separation between groups
- Improved representation of network behavior

Add:

Figure 4.1 Comparison of clustering metrics

3. Cluster Interpretation

After clustering, the original NSL-KDD labels were used only for analysis and interpretation.

The results showed that some clusters contained mostly normal traffic while others contained a larger concentration of attack records.

The clustering model does not directly classify attacks but groups traffic according to similarities in behavior.

	ID	Time	File	Algorithm	Records	Anomalies	Ratio	Silhouette	DB Index	ARI
0	4	2026-06-18T18:36:51	KDDTrain+.txt	K-Means	125973	34861	0.2767	0.4404	0.8884	0.5
1	3	2026-06-17T22:08:13	KDDTrain+_20Percent.txt	K-Means	25192	7004	0.278	0.4447	0.8422	0.5
2	2	2026-06-17T20:40:28	KDDTrain+.txt	K-Means	125973	34862	0.2767	0.4254	0.9609	0.5
3	1	2026-06-17T20:33:21	KDDTrain+_20Percent.txt	K-Means	25192	7003	0.278	0.4513	0.8582	0.5

Figure 4.2 Cluster distribution

4. Application Results

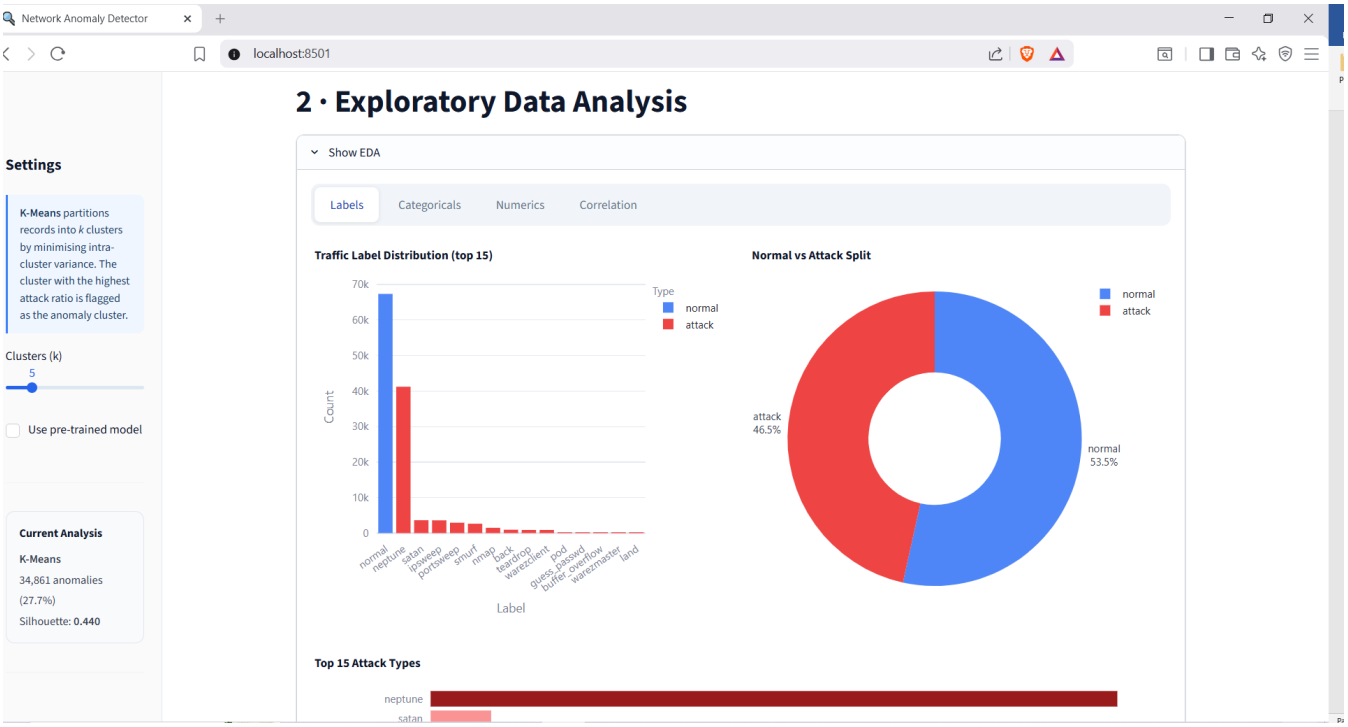
The Streamlit application successfully executed the complete workflow.

Implemented functionalities:

- Dataset upload
- Data exploration
- Preprocessing
- Clustering execution
- Metrics visualization

- Export of results

Users can upload new datasets and analyze anomalies through the graphical interface.



5. Discussion

The experimental results indicate that clustering techniques can support network anomaly detection without requiring labeled data during training.

Strengths:

- Fully unsupervised workflow
- Interactive visualization
- Reusable saved model

Limitations:

- Cluster interpretation requires post-analysis
- Performance depends on preprocessing
- NSL-KDD does not represent all modern attacks

V. Chapter 5 — Conclusion and Future Work

1. Conclusion

This project presented a complete machine learning pipeline for **Network Traffic Anomaly Detection using Clustering**. The objective was to detect unusual network behaviors automatically without relying on labeled data during training.

The implementation included dataset loading, exploratory data analysis (EDA), preprocessing, feature transformation, K-Means clustering, model evaluation, and deployment through an interactive Streamlit application.

The experiments showed that **K-Means with k = 5** achieved the best clustering performance among the tested configurations, providing better separation and representation of network traffic behavior.

The developed application allows users to upload datasets, visualize results, explore clusters, and export analysis outputs through a simple interface.

2. Achieved Objectives

The project successfully achieved the main objectives:

- Load and process the NSL-KDD dataset.
- Perform exploratory data analysis.
- Prepare data using preprocessing techniques.
- Train a clustering model for anomaly detection.
- Evaluate clustering quality using multiple metrics.
- Build an interactive Streamlit application.
- Export and save analysis results.

3. Limitations

Although the system produced satisfactory results, some limitations remain:

- K-Means requires selecting the number of clusters before training.
- Cluster interpretation requires additional analysis.
- Results depend on preprocessing quality.
- NSL-KDD may not represent all modern network threats.

4. Future Work

Future improvements may include:

- Experiment with additional clustering algorithms.
- Apply automatic parameter optimization.
- Improve feature engineering and dimensionality reduction.
- Test newer cybersecurity datasets.
- Add real-time traffic analysis.
- Deploy the application online.

5. Final Closing

This project demonstrates how unsupervised machine learning techniques can support cybersecurity by identifying unusual network behaviors and simplifying traffic analysis through an interactive and practical application.